Sense & Stop

What Member 3 (PDE and Numerical Model Lead) has done

Neuroflux team · PDE-based framework for neural wave reconstruction · Work summary, version 0.2 · 30 October 2026

In one paragraph. Member 3 built the **hidden wave** that the rest of the project tries to recover: a computer model (a partial differential equation, PDE) of a slow wave travelling across the cortex. The model was then **tuned to real rat recordings** from Member 2, so its timing and size are realistic. It was **tested** with 21 automatic checks, **documented** in a specification and an interface note, and packaged so that Members 4, 5 and 6 can use it. The work is **technically complete**; what remains is pushing it to GitHub, and a few answers from other members.

Important scope note. This is a computational study. The data are from **rats**, not humans. The model is a simplified, phenomenological one, not a physiological simulator. Nothing here shows a clinical result or a treatment for migraine.

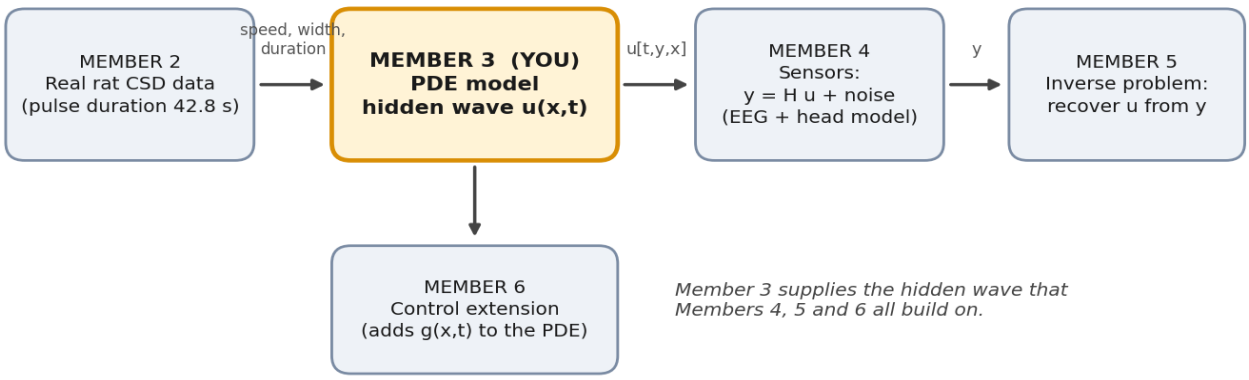
1. What Member 3 was asked to do

The methodology plan gives Member 3 five tasks and four deliverables. All are done.

Task in the plan	Status	Where
Define the state u(x,t), domain, boundary and initial conditions, and what u means	Done	model_specification.md
Implement a simple reaction-diffusion model with units, stated as a *reduced* model	Done	solver.py
Choose discretisation, time stepping, grid and boundary; keep parameters in a config file	Done	solver.py, model_default.yaml
Connect model scale to Member 2's data; say where values are illustrative	Done (preliminary data)	calibration.py, spec Sec. 6-7
Give Members 4 and 5 a stable interface returning u(x,t) on a named grid	Done	interface_note_model.md

Deliverables: model specification, solver module, configuration file, interface note. In addition: a calibration tool and a test suite.

2. Where this fits in the team pipeline



Member 2 supplies real measurements of how long the wave lasts. Member 3 turns that into a model of the wave. Member 4 simulates what sensors would see; Member 5 tries to recover the

wave from those sensors; Member 6 later adds a hypothetical control term. If the hidden wave is wrong or badly documented, everything downstream suffers, which is why the **interface note** is the most urgent deliverable.

3. What was built

File	What it does
configs/model_default.yaml	Every parameter in one place, with its unit and a provenance tag: measured, literature-derived, calibrated, design choice, or illustrative.
src/model/solver.py	`simulate(config)` runs the model in 1D or 2D with three time-steppers (Euler, RK4, scipy solve_ivp), an automatic stability guard, no-flux boundaries, an optional control hook for Member 6, and save/load of results with metadata.
src/model/calibration.py	`calibrate_excitable()` computes the model's D and k from a target pulse duration and speed. `pulse_metrics()` measures duration, rise and decay the same way Member 2 did.
tests/test_model.py	21 automatic checks (numerical correctness, agreement with the real data, calibration, interface).
docs/model_specification.md	Equations, assumptions, parameter provenance, comparison to data, verification, limitations, open decisions.
docs/interface_note_model.md	Array shapes, units, rules that Members 4, 5, 6 must respect, and questions for Member 4.

4. The two models

Both models describe a normalised state **u**: 0 means resting, higher means "depolarised". Using a normalised state means the unknown amplifier gain in the rat recordings does not matter.

Excitable model (primary): a wave that rises and then recovers

$$u_t = D\,\nabla^2 u + k\,[\,u(1-u)(u-a) - v\,]$$

$$v_t = k\,\varepsilon\,(\,u - \gamma\,v\,)$$

A second variable v slowly builds up and shuts the wave off behind it. This produces a **travelling pulse with recovery**, which is what the literature review says a CSD-like model needs.

Fisher–KPP (baseline): a front that only advances

$$u_t = D\,\nabla^2 u + r\,u(1-u)$$

A classic invasion model. It is simple and has exact mathematical properties we can test against, but it has **no recovery**: once the wave arrives, the state stays elevated. So it is kept only as a front-reconstruction baseline.

> **A team decision was needed here.** The methodology plan names only Fisher–KPP, but the literature review (Sec. 2.2.2) recommends a two-variable excitable model as the primary system. Both are implemented behind the same interface, and the excitable model is now the default. Member 1 should log this decision (proposed as D7).

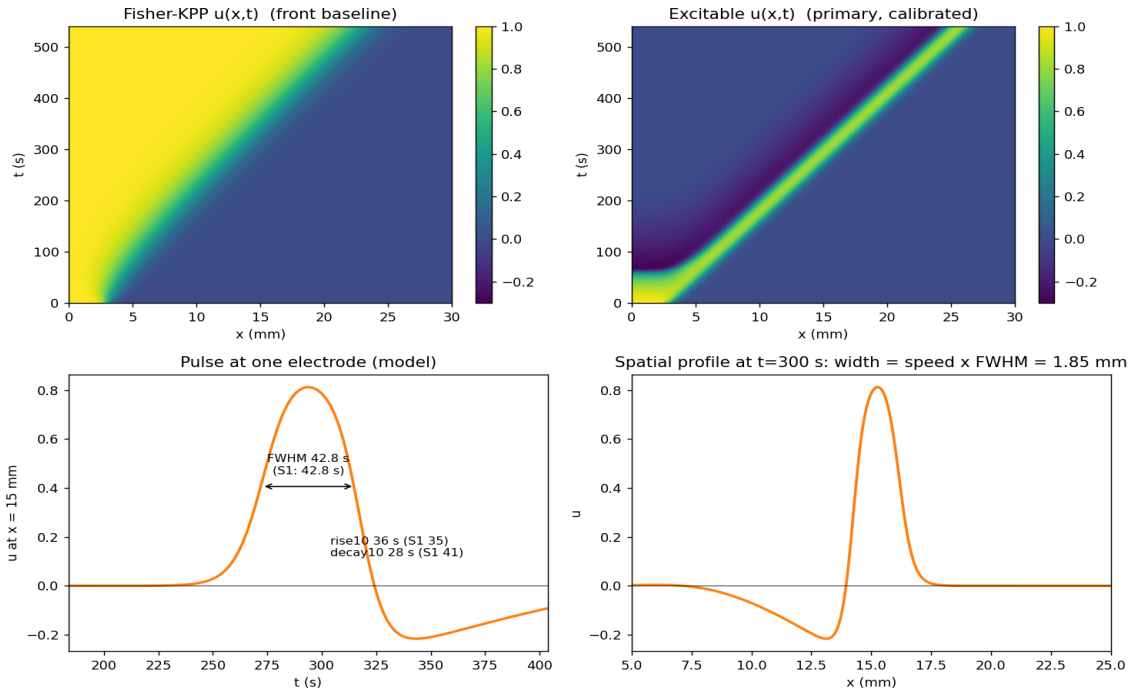


Figure 1. Top: space-time maps. Fisher–KPP (left) advances and stays elevated; the excitable model (right) is a pulse followed by recovery. Bottom: the pulse as one electrode would see it (left) and its shape in space (right).

5. Tying the model to real data

What Member 2 provided (rat, 3 animals, 15 waves, preliminary):

Quantity	Value	Basis
Pulse duration at one electrode (FWHM)	42.8 s (animal means 36 to 56 s)	**Measured**
Rise time (10% to peak) / decay time (peak to 10%)	35.2 s / 40.7 s	**Measured**
Front speed	2.3 to 2.9 mm/min (narrow), 2.0 to 9.0 mm/min (broad)	**Literature**, not yet verified by reading the papers
Pulse width	1.7 to 2.1 mm	Derived: speed × FWHM
Domain size, 1D or 2D geometry	Not available	One electrode cannot give spatial information

How the calibration works, in plain words.

- The model has a scaling property: **time** scales with 1/k and **length** scales with √(D/k).
- So the pulse is simulated once in dimensionless form (D = k = 1) to measure its duration T_0 = 21.56 and speed c_0 = 0.448.
- Then k = T_0 / (target duration) and D = (target speed / c_0)^2 / k. This gives **k = 0.504 s^{-1}** and **D = 0.0186 mm^2/s**.
- The tool `calibrate_excitable()` does this in one call, so when Member 2's numbers change, nothing has to be re-tuned by hand.

How well the calibrated model matches the data:

Feature	Real data	Model	Is it an independent check?
Pulse duration (FWHM)	42.8 s	42.8 s	No, it is the calibration target
Speed	2.3 to 2.9 mm/min	2.60 mm/min	No, calibration target
Pulse width	1.7 to 2.1 mm	1.85 mm	No, follows from the two above
Rise time (10% to peak)	35.2 s	36.3 s	**Yes: agrees**
Decay time (peak to 10%)	40.7 s (28 to 47)	28.2 s	**Yes: at the low edge, model recovers too fast**

Only the last two rows are real checks. With three animals (and animal R6 differing from R4), this is a **plausibility check, not validation**.

Final parameter values

Parameter	Value	Basis
D, k (excitable)	0.018561 mm^2/s, 0.503758 s^{-1}	Calibrated
a, ε, γ (shape of the pulse)	0.1, 0.01, 2.0	Design choice (γ chosen so the pulse shape resembles the data)
D, r (Fisher–KPP)	0.01296 mm^2/s, 0.036 s^{-1}	Calibrated by hand: speed 2.6 mm/min
Grid spacing	0.1 mm (must be ≤ 0.096 mm to resolve the front)	Design choice, convergence-tested
Domain	30 mm (1D) or 30 × 20 mm (2D)	**Design choice, to be agreed with Member 4**
Run length / output spacing	540 s / every 2 s	Design choice

6. Problems found along the way, and how they were fixed

Part of the value of this work is what went wrong and was caught. These are recorded openly.

What happened	What was done
The first model's pulse lasted **165 s** and was **6.5 mm** wide, about 4 times too long and wide compared with the real 43 s.	Re-calibrated to the real data (Section 5). Grid refined to 0.1 mm.
My first excitable parameters collapsed; later, many parameter sets made the wave die out.	Scanned parameters systematically; found a stable set; documented that the pulse is fragile (ε ≥ 0.015 or a ≥ 0.15 killed it).
I expected the Fisher–KPP front speed to equal 2√(Dr). It measured about 25% lower.	This is a known property of such fronts: speed approaches the limit only slowly, from below. Replaced the test by an **exact analytical solution**.
The exact-solution error grew after the grid was made sharper (4.9e-4).	Checked rather than loosened the tolerance: error falls by exactly **4.00×** when the grid is halved, which is textbook second order. The solver was right.
A comment in the config claimed small stimuli kill the wave. A test showed it was false.	Corrected to what was actually observed.
The excitable state dips **below rest** (about −0.3) after the pulse.	Documented. Members 4 and 5 must not clip u to [0, 1] for this model.

7. How we know it works (21 automatic tests, all passing)

Check	What it proves	Result
Exact Fisher–KPP travelling wave	The solver solves the equation correctly	Error 4.9e-4; drops 4.00× when dx halves; speed within 0.2%
Grid refinement (space)	Accuracy improves at the expected rate	Second order
Time-step refinement (Euler)	Time stepping behaves correctly	First order
RK4 vs scipy solve_ivp	Two independent solvers agree (needed to avoid the "inverse crime")	Max difference below 5e-3
No-flux boundary, pure diffusion	Nothing leaks through the walls	Mass conserved to 1e-10
2D planar wave vs 1D	2D code is consistent with 1D	Identical to 1e-10
Stability guard	Too-large time steps are reduced automatically	No blow-up
Excitable pulse vs data	Timing and speed match the S1 features	FWHM within 3%; speed 2.3 to 2.9 mm/min
Grid convergence of the pulse	Default grid is fine enough	Halving dx changes results by less than 2%
Measurement functions	FWHM, rise, decay measured correctly	Exact on a Gaussian test pulse
Calibration tool	A different target (50 s, 4 mm/min) is reproduced	Within 5%; errors out if no pulse exists
Control hook	Zero control changes nothing; strong control blocks the wave	Passes
Known mismatch (decay time)	The 28 s vs 41 s gap is recorded on purpose as a test	Documented

Sensitivity scenarios (literature broad range, checked by simulation): at 2.0 mm/min the model gives 1.99 mm/min and a 1.42 mm pulse; at 9.0 mm/min it gives 9.27 mm/min and a 6.71 mm pulse (about 3% to 5% off its targets). The wave is sustained in all three cases. The 2D versions were not run.

8. Limitations

- **Thin data.** Three rat animals, preliminary; R6 differs from R4. The speed comes from literature and is not yet verified, because the recordings have only one electrode.
- **Decay is too fast** in the model (28 s against 41 s in the data).
- **Undershoot.** After the pulse, u falls to about −0.2 to −0.3 and relaxes over a couple of minutes. The data do not constrain this; it is typical of this model type, not a physiological prediction.
- **Fragile pulse.** The excitable pulse is sensitive to a, ε and γ; always re-run the tests after changing them.
- **Reduced model.** No ions, glia, metabolism or blood flow; a flat rectangular domain with no folding.
- **Domain size is a design choice,** not data. Fisher–KPP has no recovery at all.

9. What is still open

Item	Who	Effort
Push to GitHub on a `member3/...` branch, open a pull request, open an interface issue, add decision-log entries D6 (update) and D7	Member 3	~1 hour
Have another member review the pull request (required by CONTRIBUTING.md)	Reviewer	Short
Confirm domain size, source spacing, and 2D (the pulse is 1.85 mm wide, probably finer than the source spacing, so H should act on an area-averaged field)	Member 4	5 to 10 minutes
Verify the two speed papers; add animals if possible. Then re-run `calibrate_excitable()` and the tests	Member 2, then Member 3	Under 1 hour
Log the primary-model decision (excitable)	Member 1	5 minutes
Define the control term, its bounds, objective and the search (the hook already exists)	Members 3 and 6	After the pull request merges

10. How to run it

```
pip install numpy scipy pyyaml pytest matplotlib      # numpy 2.0 or newer
python -m pytest tests/ -q                            # expect: 21 passed

from src.model import simulate, load_config
cfg = load_config("configs/model_default.yaml")
res = simulate(cfg)            # res["u"] has shape [t, x]  (2D: [t, y, x]); units mm and s
```

Default runs: 1D about 1 second; 2D (30 × 20 mm) about 33 seconds and 131 MB for u. Use `python -m pytest`, not bare `pytest`, so the imports work.